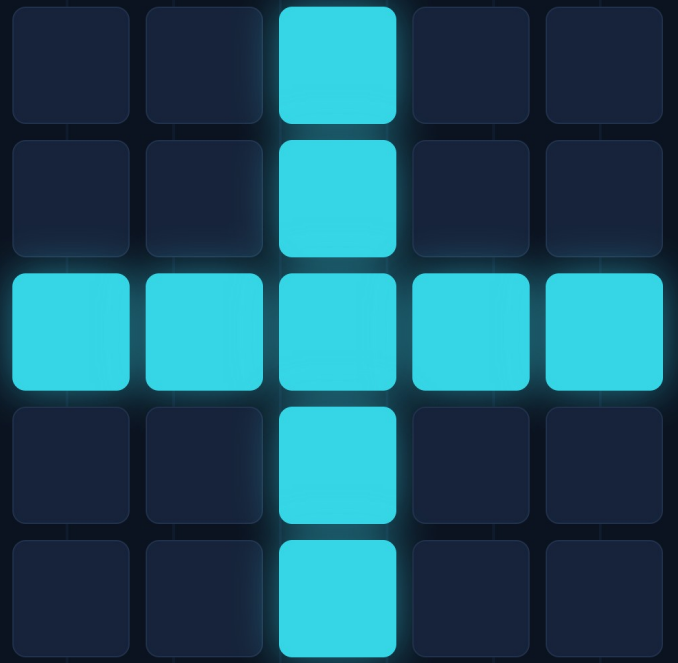
CODYSSEY 미션 3 · 쉽게 배우기

컴퓨터는 어떻게 모양을 알아볼까?

작은 AI 계산기 **Mini NPU** 만들기



한눈에 보기

이번 미션은 이 4단계를 코드로 만드는 거예요

01
그림
십자가(+)나 X 모양 그림이 들어와요

→

02
숫자
칠한 칸은 1, 빈 칸은 0으로 바꿔요

→

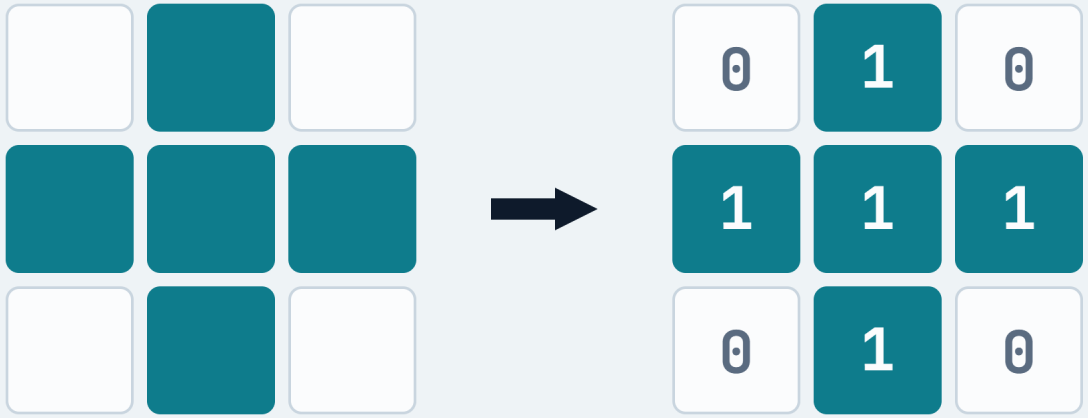
03
점수
본보기 카드와 곱하고 더해요

→

04
판정
점수가 높은 쪽이 정답이에요

3단계의 **'곱하고 더하기'**가 AI 칩이 하루 종일 하는 일이에요.

컴퓨터 눈에는 그림이 아니라 숫자만 보여요



칠한 칸 = 1, 빈 칸 = 0

이렇게 숫자로 만든 표를 **2차원 배열**이라고 해요.

파이썬에서는 **리스트 안에 리스트**를 넣어서 만들어요.

```
[[0,1,0], [1,1,1], [0,1,0]]
```

필터 = 찾고 싶은 모양의 '본보기 카드'



십자가 필터



X 필터

숨은그림찾기 할 때 손에 든 **정답 카드**처럼, 컴퓨터는 이 카드를 입력 그림 위에 **겹쳐 봐요.**

> 실제 data.json 필터에는 0.9, 0.3 같은 숫자도 있어요. 숫자가 클수록 **그 칸이 더 중요하다**는 뜻이에요.

MAC = 같은 칸끼리 곱하고, 모두 더하기

십자가 입력
× 십자가 필터



→ **점수 5**

십자가 입력
× X 필터



→ **점수 1**

Multiply(곱하기) + **A**ccumulate(쌓아서 더하기) = **MAC** · 0 × 무엇이든 = 0 이라서 **겹치는 칸만 살아남아요.**

3단계 · 판정

점수가 더 높은 필터가 이겨요

십자가 필터

5

승리!

VS

X 필터

1

아쉽게 패배

그래서 결과는 **십자가**!

그런데 점수가 **똑같으면?**
→ **판정 불가** (영어로 UNDECIDED, 무승부)

왜 NPU가 필요할까?

AI는 이 계산을 정말 정말 많이 해요

9번

3×3 필터 한 번

625번

25×25 필터 한 번

수억 번

진짜 AI · 사진 한 장

CPU = 선생님 1명

시험지를 **한 장씩 차례로** 채점해요. 똑똑하지만 1만 장이면 오래 걸려요. (직렬)

NPU = 조교 1,000명

O, X만 보는 단순한 일을 **동시에** 해요. 그래서 훨씬 빨라요. (병렬)

우리가 만들 프로그램 · main.py

핵심 메뉴 2개: 직접 입력, 그리고 문제집 채점

모드 1

키보드로 직접 입력

3×3 필터 A, B와 그림을 한 줄씩 입력하면 **점수 · 계산 시간 · 판정**을 보여 줘요.

```
0 a 0 → "숫자만 입력하세요" → 그 줄만 다시
```

모드 2

data.json 문제집 채점

5×5, 13×13, 25×25 문제를 한꺼번에 풀고 **PASS / FAIL, 속도표, 성적표**를 보여 줘요.

```
총 6개 · 통과 3 · 실패 3
```

약속: NumPy 같은 도우미 도구 없이 **for 반복문으로 직접** 계산해요. (메뉴 3은 보너스 속도 비교, 0은 종료)

data.json

data.json은 컴퓨터용 '문제집'이에요

```
"filters": {
"size_13": { "cross": [...], "x": [...] }
},
"patterns": {
"size_13_2": {
"input": [[...]],
"expected": "+"
}
}
```

문제 이름에 **크기**가 숨어 있어요.

`size` **`13`** `2`

13 = 13×13 크기, **2** = 2번 문제
→ 그래서 **size_13 필터**를 꺼내 써요.

input = 문제 그림, expected = 정답

> 크기가 안 맞는 문제가 있어도 **그 문제만 FAIL**, 프로그램은 멈추지 않아요.

라벨 정규화

채점 전에 이름표부터 통일해요

+	→	Cross
cross	→	Cross
X	→	X

‘Cross’와 ‘+’는 뜻은 같지만, 컴퓨터에겐 **완전히 다른 글자**예요.

출석부에 친구 이름을 ‘민수’, ‘김민수’, ‘minsu’로 제각각 쓰면 대조가 안 되죠? 그래서 **읽자마자 하나로 맞춰요.**

공백 지우기 → 소문자로 → 표준 이름

컴퓨터도 계산 실수를 한다고?!

```
0.1 + 0.2 = 0.30000000000000004
for문으로 0.1을 9번 더하기 = 0.8999999999999999
```

※ 우리 MAC 함수처럼 total += 로 하나씩 더할 때 그래요. 파이썬 3.12부터 sum()은 오차를 보정해서 0.9가 나와요.

왜 그럴까?

1 ÷ 3 = 0.3333…이 끝없이 이어지듯, 0.1도 컴퓨터 숫자(2진수)로 바꾸면 **끝이 없어서** 살짝 잘린 값으로 저장돼요.

무엇이 문제일까?

여러 번 더할수록 **작은 오차가 쌓여요.** 그래서 수학으로는 같은 점수인데, 컴퓨터에선 아주 조금 달라져요.

차이가 아주아주 작으면 ‘무승부’로 정해요

```
|A - B| < 0.000000001 → 무승부
```

저울 두 개가 1.000kg과 0.999999kg을 가리키면 ‘같은 무게’라고 보는 게 맞죠? 그 ‘이 정도는 같다’는 기준이 **epsilon**(1e-9)이에요.

실제 data.json 채점 결과

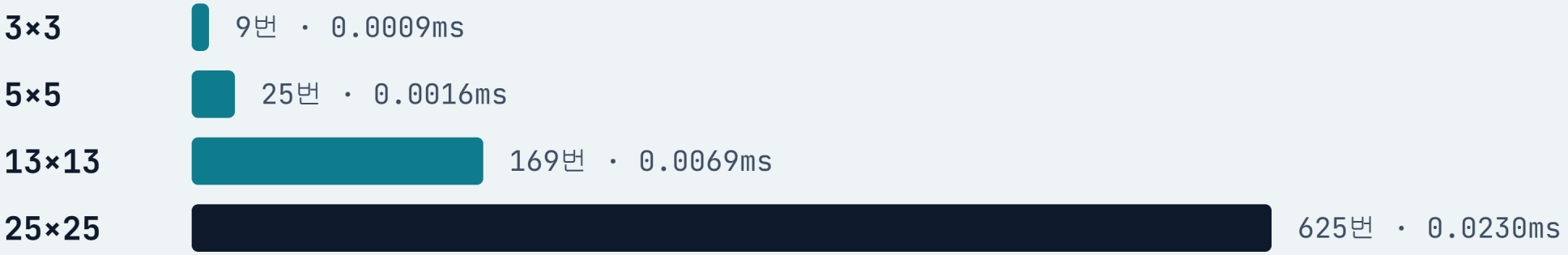
6문제 중 3개 통과, 3개는 무승부

문제	Cross 점수	X 점수	판정	결과
size_5_1	0.9	0.8999999999999999	UNDECIDED	FAIL
size_5_2	8.9	0.1	Cross	PASS
size_13_1	0.3	14.700000000000008	X	PASS
size_13_2	7.499999999999997	7.5	UNDECIDED	FAIL
size_25_1	4.9	4.899999999999999	UNDECIDED	FAIL
size_25_2	52.9	0.1	Cross	PASS

> UNDECIDED = 무승부. 실패 3개는 **버그가 아니에요!** 수학으로는 똑같은 점수(0.9 = 0.1×9)라서 무승부 규칙이 **제대로 작동한 결과**예요.

성능 분석 · 10번 재서 평균

그림이 커지면 계산은 얼마나 늘까?



N×N 칸을 하나씩 다 계산하니까 **계산 횟수 = N × N**

한 변이 **2배**면 계산은 **4배** → 이걸 **O(N²)**이라고 불러요

실패 원인 찾기

FAIL이 나오면 탐정처럼 3곳을 살펴봐요

① 문제지가 이상해?

이름 규칙, 크기, 정답 표시가 약속대로인지 (데이터)

이번엔 0건

② 계산법이 틀렸어?

곱하기·더하기 코드가 맞는지 (로직)

이번엔 0건

③ 비교 방법이 문제?

계산 오차 때문에 무승부가 됐는지 (수치 비교)

이번엔 3건

무승부가 아닌 3문제가 모두 정답 → **계산 코드는 정상**이라는 증거예요.

보너스 과제

보너스 2가지도 만들었어요

① 한 줄로 펼치기



표를 **한 줄 리스트**로 펴면 칸 찾기가 한 번으로 줄어 **대략 15~40% 빨랐어요**(잴 때마다 달라요). 하지만 계산 횟수는 그대로 N^2예요.

② 모양 자동 생성기

```
make_cross(25) make_x(25)
```

크기 N만 넣으면 그림을 만들어 줘요.
십자가 = 가운데 가로줄 + 세로줄이 1
X = 두 대각선이 1

과제 목표 체크

이 6가지를 내 말로 설명하면 성공!

① MAC이 뭐야?

같은 칸끼리 곱해서 다 더한 점수. AI는 이걸 수억 번 해서 NPU가 필요해.

② 점수로 닮음을 어떻게 알아?

겹치는 칸만 살아남으니, 많이 겹칠수록 점수가 커져.

③ 왜 이름표를 통일해?

‘+’와 ‘Cross’는 다른 글자라, 맞춰야 채점이 맞아.

④ 왜 무승부 규칙이 필요해?

0.1 같은 수는 오차가 생겨서, 아주 작은 차이는 같다고 봐야 해.

⑤ 크기가 커지면?

N×N칸을 다 계산하니 한 변 2배면 4배. 이게 $O(N^2)$.

⑥ 틀리면 어디부터 봐?

문제지(데이터) → 계산 코드(로직) → 비교 방법 순서로.

한 문장 정리

AI가 '본다'는 건 숫자끼리 곱하고 더하는 것이에요

오늘 만든 Mini NPU가 바로 그 원리예요.

